

Linkro – Full Project Documentation (Detailed Version)

Linkro is a full-stack MERN platform designed to connect recruiters with potential agents, streamline hiring workflows, and deliver an admin-controlled ecosystem.

This document captures:

- Full architecture overview
- Folder structure blueprint
- Feature definitions
- API design overview
- Data flow summaries
- Setup & deployment guide

1. TECHNOLOGY STACK

FRONTEND:

- React + Vite
- Tailwind CSS v4
- React Router v6
- Axios
- React Query (optional)
- JWT Decode

BACKEND (planned):

- Node.js + Express
- MongoDB + Mongoose
- Redis (optional caching)
- AWS S3 (resume uploads)
- Socket.io (real-time chat)

DEPLOYMENT:

- Frontend → Vercel
- Backend → Render / AWS EC2
- Reverse Proxy → Nginx

2. FRONTEND FOLDER STRUCTURE

src/

api/

client.js

auth.js

agent.js

recruiter.js

admin.js

components/

Sidebar.jsx

Navbar.jsx

Button.jsx

Loader.jsx

Modal.jsx

context/

AuthContext.jsx

hooks/

useAuth.js

useFetch.js

useToggle.js

layouts/

AuthLayout.jsx

DashboardLayout.jsx

AdminLayout.jsx

pages/

auth/

Login.jsx

Register.jsx

ForgotPassword.jsx

ResetPassword.jsx

agent/

AgentDashboard.jsx

AgentJobs.jsx

AgentJobDetails.jsx

AgentApplications.jsx

AgentApplicationDetails.jsx

AgentProfile.jsx

AgentChat.jsx

AgentSettings.jsx

recruiter/

RecruiterDashboard.jsx

RecruiterJobs.jsx

RecruiterJobDetails.jsx

RecruiterCreateJob.jsx

RecruiterApplications.jsx

RecruiterApplicationDetails.jsx

RecruiterChat.jsx

RecruiterProfile.jsx

admin/

AdminDashboard.jsx

ManageUsers.jsx

ManageUserDetails.jsx

ManageJobs.jsx

ManageJobDetails.jsx

SystemAnalytics.jsx

SystemLogs.jsx

routes/

ProtectedRoute.jsx

RoleRoute.jsx

utils/

helpers.js

constants.js
formatDate.js

App.jsx

main.jsx

index.css

3. FEATURE OVERVIEW

AUTH SYSTEM:

- Login, Register, Forgot Password, Reset Password
- Role-based access control
- JWT authentication
- Protected routes + Role routes

AGENT FEATURES:

- Explore job listings
- Apply to jobs
- Track application status
- Chat with recruiters
- Manage profile & settings

RECRUITER FEATURES:

- Post job listings
- Manage applications
- View agent profiles
- Chat with agents
- Edit job details

ADMIN FEATURES:

- Manage all users
- Manage job listings
- View system logs
- View analytics dashboard

REAL-TIME SYSTEM (planned):

- Agent ↔ Recruiter chat
- Application status notifications

4. DATA FLOW OVERVIEW

AUTH FLOW

User → Auth API → JWT Issued → ProtectedRoute → RoleRoute → Dashboard

AGENT FLOW

Agent → Browse Jobs → Apply → Application_DB → Recruiter Notification

RECRUITER FLOW

Recruiter → Create Job → Job_DB → Agent Browsing → Recruiter Reviews Applications

ADMIN FLOW

Admin → Manage Users/Jobs → Update Databases → System Monitoring

CHAT FLOW

User → Socket.io → Real-Time Channel → Message_DB → Notification Event

5. API OVERVIEW (PLANNED BACKEND)

AUTH:

POST /auth/register

POST /auth/login

POST /auth/forgot-password

POST /auth/reset-password

AGENT:

GET /agent/jobs

GET /agent/applications

POST /agent/apply/:jobId

RECRUITER:

POST /recruiter/jobs

GET /recruiter/jobs

GET /recruiter/applications

ADMIN:

GET /admin/users

GET /admin/jobs

DELETE /admin/user/:id

DELETE /admin/job/:id

6. PROJECT SETUP

INSTALL DEPENDENCIES:

```
npm install
```

RUN DEVELOPMENT:

```
npm run dev
```

ENV FILE:

```
VITE_API_URL=http://localhost:5000/api
```

7. DEPLOYMENT WORKFLOW

1. Build frontend using Vercel
2. Deploy backend on Render or AWS EC2
3. Configure reverse proxy on Nginx
4. Connect domain + SSL (HTTPS)
5. Set up monitoring (CloudWatch, Grafana optional)

8. NEXT DEVELOPMENT STEPS

- Implement routing for all roles
- Build dashboard UI
- Connect auth pages to backend
- Initialize backend project structure
- Implement job and application APIs
- Build chat service using Socket.io
- Add admin analytics